

Document Number: P3324R0
Date: 2024-10-14
Audience: EWG
Reply-to: Tom Honermann <tom@honermann.net>

Attributes for namespace aliases, template parameters, and lambda captures

- [Introduction](#)
- [Motivation](#)
- [Design considerations](#)
 - [Namespace alias definitions](#)
 - [Template parameter declarations](#)
 - [Lambda captures](#)
- [Proposal](#)
- [Implementation experience](#)
- [Acknowledgements](#)
- [References](#)
- [Wording](#)

Introduction

As evidenced by [annex A \(Grammar summary\)](#) in the C++ standard, an attribute may appertain to almost every kind of declared name or entity including:

- Type aliases
- Enumerations
- Enumerators
- Variables (including the unnamed object for structured bindings)
- Structured bindings
- Classes
- Data members
- Functions and function types
- Parameters (function parameters, non-type template parameters, exception handlers)
- Concepts
- Labels
- Namespaces
- Modules

Exceptions include:

- Namespace aliases
- Type template parameters
- Template template parameters
- Lambda captures

It is not clear why attributes are not supported for the items in the exceptions list. There are existing standard attributes that appear to be relevant to these items based on their specification or stated recommended practice.

The recommended practice for the `[[deprecated]]` attribute in [\[dcl.attr.deprecated\]p4](#) seems applicable to namespace aliases:

Recommended practice: Implementations should use the deprecated attribute to produce a diagnostic message in case the program refers to a name or entity other than to declare it, after a declaration that specifies the attribute. The diagnostic message should include the text provided within the [attribute-argument-clause](#) of

any deprecated attribute applied to the name or entity. The value of a [has-attribute-expression](#) for the deprecated attribute should be 0 unless the implementation can issue such diagnostic messages.

The recommended practice for the `[[maybe_unused]]` attribute in [\[dcl.attr.unused\]p4](#) seems applicable to template parameters and lambda captures.

Recommended practice: For an entity marked `maybe_unused`, implementations should not emit a warning that the entity or its structured bindings (if any) are used or unused. For a structured binding declaration not marked `maybe_unused`, implementations should not emit such a warning unless all of its structured bindings are unused. For a label to which `maybe_unused` is applied, implementations should not emit a warning that the label is used or unused. The value of a [has-attribute-expression](#) for the `maybe_unused` attribute should be 0 if the attribute does not cause suppression of such warnings.

The `[[no_unique_address]]` attribute applies to non-static data members and therefore seems applicable to unnamed non-static data members implicitly declared in lambda closure types for captured entities. From [\[dcl.attr.nouniqueaddr\]p1](#):

The [attribute-token](#) `no_unique_address` specifies that a non-static data member is a potentially-overlapping subobject ([\[intro.object\]](#)). No [attribute-argument-clause](#) shall be present. The attribute may appertain to a non-static data member other than a bit-field.

The `[[deprecated]]` attribute was adopted for C++14 via [N3760](#) [\[N3760\]](#). Neither that paper, its previous revision, nor the minutes from EWG and CWG review during the following meetings demonstrate consideration for the use of the attribute with a namespace alias.

- [EWG review of N3394 in Portland, 2012](#)
- [CWG review of N3760 in Chicago, 2013](#)

The `[[maybe_unused]]` attribute was adopted for C++17 via [P0212R1](#) [\[P0212R1\]](#). Neither that paper, its previous revision, the rationale provided in the previous [P0068R0](#) [\[P0068R0\]](#), nor the minutes from EWG and CWG review during the following meetings demonstrate consideration for use of the attribute with a type template parameter, template template parameter, or lambda capture.

- [EWG review of P0068R0 in Kona, 2015](#)
- [CWG review of D0212R1 in Jacksonville, 2016](#)

The `[[no_unique_address]]` attribute was adopted for C++20 via [P0840R2](#) [\[P0840R2\]](#). Neither that paper, the motivation provided in its original R0 revision, nor the minutes from EWG and CWG review during the following meetings demonstrate consideration for the use of the attribute with a lambda closure.

- [EWG review of P0840R0 in Albuquerque, 2017](#)
- [CWG review of P0840R1 and D0840R2 in Jacksonville, 2018](#)

A search of both active and closed CWG issues failed to identify any existing issues that concern attribute appertinment for the excluded items.

There does not appear to be a recorded rationale for omitting attribute support for the excluded items.

The C++ grammar permits an optional sequence of attributes in the declaration of a non-type template parameter by way of [template-parameter](#) and [parameter-declaration](#). The latter directly permits an attribute sequence at the beginning of a parameter declaration and an attribute sequence is indirectly permitted following a parameter name by [nopr-declarator](#). However, some implementations, Clang and EDG among them, diagnose an attribute that appertains to a non-type template parameter in at least some cases. The gcc and Microsoft Visual C++ compilers accept such attributes. <https://godbolt.org/z/G61qK7oPc>.

```
template<[[maybe_unused]] int nttp> // Rejected by EDG and Clang, accepted by gcc and MSVC.
void ft1();
template<int nttp [[maybe_unused]]> // Rejected by EDG, accepted by Clang, gcc, and MSVC.
void ft2();
```

Motivation

Per [\[dcl.attr.deprecated\]p2](#), the deprecated attribute is explicitly specified as being able to appertain to a namespace.

The attribute may be applied to the declaration of a class, a [typedef-name](#), a variable, a non-static data member, a function, a namespace, an enumeration, an enumerator, a concept, or a template specialization.

The inability to apply the deprecated attribute to a namespace alias seems to be an oversight as there is no principled reason for namespace aliases to never be deprecated. At present, Clang, gcc, EDG, and Microsoft Visual C++ all reject an attribute sequence on a namespace alias either before the identifier at a position consistent with attributes on namespace declarations, or after the identifier. <https://godbolt.org/z/8oqah7T15>

```
namespace ns {}
namespace [[deprecated]] ns1 = ns; // Rejected by Clang, EDG, gcc, and MSVC.
namespace ns2 [[deprecated]] = ns; // Rejected by Clang, EDG, gcc, and MSVC.
```

Per [\[dcl.attr.unused\]p2](#), the maybe_unused attribute is specified as applicable to non-static data members.

The attribute may be applied to the declaration of a class, [typedef-name](#), variable (including a structured binding declaration), structured binding, non-static data member, function, enumeration, or enumerator, or to an [identifier](#) label ([\[stmt.label\]](#)).

Likewise for the no_unique_address attribute per [\[dcl.attr.nuniqueaddr\]p1](#):

The [attribute-token](#) no_unique_address specifies that a non-static data member is a potentially-overlapping subobject ([\[intro.object\]](#)). No [attribute-argument-clause](#) shall be present. The attribute may appertain to a non-static data member other than a bit-field.

A lambda capture is not formally a non-static data member, though references to a lambda capture are usually translated to an access of a non-static data member of the closure type. As such, most attributes that may appertain to a non-static data member are likely to be applicable to lambda captures. For example, the maybe_unused attribute could be used to mark a lambda capture as intentionally unused. Such unused captures can be useful to acquire resources such as locks or to preserve the layout of a closure type when a previously needed capture is no longer needed. Likewise, the no_unique_address attribute could be used to request a more optimal class layout for a lambda closure type.

At present, Clang issues a warning under `-Wall` for unused lambda captures. <http://eel.is/c++draft/stmt.label>. Clang, gcc, EDG, and Microsoft Visual C++ all reject the following code. <https://godbolt.org/z/4xq1YKodz>

```
namespace std {
    struct mutex {};
    template<typename... Ts>
    struct scoped_lock {
        scoped_lock(Ts...);
    };
}

void f(int p1, int p2) {
    std::mutex m1, m2;
    [ [[maybe_unused]] p1 ]{}();
    [ p2 [[maybe_unused]] ]{}();
    [ [[maybe_unused]] l1 = std::scoped_lock(m1) ]{}();
    [ l2 [[maybe_unused]] = std::scoped_lock(m2) ]{}();
}
```

None of Clang, gcc, EDG, or Microsoft Visual C++ currently issue warnings for unused template parameters and this is unlikely to change given how frequently template parameters are unused. However, for consistency and to allow for explicit documentation, it is reasonable to allow the maybe_unused attribute to appertain to all kinds of template parameters. <https://godbolt.org/z/YbE9cTPvh>

```
template<[[maybe_unused]] int nttp> // Rejected by EDG and Clang, accepted by gcc and MSVC.
```

```

void ft1();
template<int nttp [[maybe_unused]]> // Rejected by EDG, accepted by Clang, gcc, and MSVC.
void ft2();
template<[[maybe_unused]] typename ttp> // Rejected by Clang, EDG, gcc, and MSVC.
void ft3();
template<typename ttp [[maybe_unused]]> // Rejected by Clang, EDG, gcc, and MSVC.
void ft4();
template<[[maybe_unused]] template<typename> class tttp> // Rejected by Clang, EDG, gcc, and MSVC.
void ft5();
template<template<typename> class tttp [[maybe_unused]]> // Rejected by Clang, EDG, gcc, and MSVC.
void ft6();

```

Absent compelling rationale otherwise, precedent indicates that new kinds of declared names or entities should allow for attribute appertinment and their respective declarations should therefore include syntactic locations for an attribute specifier sequence.

Design considerations

There are various and conflicting precedents for where an attribute specifier sequence is syntactically placed in existing declarations.

1. In general, an attribute specifier sequence immediately follows the identifier in a declaration for an attributed entity. Examples include declarations of type aliases, enumerators, variables, structured bindings, data members, functions, parameters, concepts, and modules. Exceptions include label, class, and enum declarations and namespace definitions.
2. [namespace-definition](#) does not permit an attribute specifier sequence following an identifier (inconsistent with the general case) but does permit one following the namespace keyword (presumably to allow an attribute to be specified for an unnamed namespace definition).
3. [parameter-declaration](#) permits an attribute specifier sequence following a declarator ID (consistent with the general case) and at the start of a parameter declaration (presumably to allow an attribute to be specified for an unnamed parameter declaration).

Namespace alias definitions

Namespace alias definitions always include an identifier. This presents a choice of:

1. following the precedent for namespace definitions and allowing an attribute specifier sequence between the namespace keyword and the identifier, or
2. following the general precedent and allowing an attribute specifier sequence following the identifier, or
3. both.

No motivation for supporting both has been identified. Option 1 is proposed based on the rationale that consistency with namespace definitions is more important than consistency with a general rule that already has existing exceptions.

Template parameter declarations

Type template parameters and template template parameters may be unnamed. This presents a choice of:

1. following the precedent for non-type template parameters and function parameters and allowing an attribute specifier sequence both at the beginning of a declaration and following an identifier, or
2. allowing an attribute specifier sequence only at the beginning of a declaration, or
3. allowing an attribute specifier sequence following an identifier with no provision for specifying attributes for unnamed parameters.

Option 1 is proposed based on the rationale that consistency with non-type template parameters and function parameters is more important than avoiding the cognitive and implementation complexity of supporting multiple locations.

Lambda captures

A lambda capture serves two purposes; it grants access to a local variable from within a lambda expression and it notionally declares a non-static data member of the closure type. An attribute applied to a lambda capture can therefore be considered to appertain to either the capture itself or to the notional non-static data member.

Consider the two lambda expressions in the following example. The first, `l1`, explicitly captures `i` but does not use it. The second, `l2`, has a default capture, but no variables are implicitly captured. Depending on one's intuition, the attribute might be intended to suppress a diagnostic about an unused capture as opposed to an unused capture declaration.

```
void f(int i) {
    // Don't warn that 'i' is captured but not used?
    auto l1 [i [[maybe_unused]] ] {
        return 0;
    };
    // Don't warn that the default capture declaration was not used?
    auto l2 [= [[maybe_unused]] ] {
        return 0;
    };
}
```

This proposal specifies that the attribute appertains to the notional non-static data member on the basis that doing so is more useful. For example, `[[no_unique_address]]` has a clear and unambiguous meaning for the notional non-static data member, but would otherwise be meaningless for a capture declaration. As explained below, this proposal locates the new attribute specifier sequence following each capture declaration. Should motivation arise for attributes that appertain directly to the capture declaration, it would be possible to add such support by additionally allowing for an attribute specifier sequence to precede each capture declaration.

Lambda capture lists are similar to parameter declaration lists but differ in that implicitly captured entities are not individually declared; there is no notion of an unnamed capture in the same sense as an unnamed function parameter. This presents a choice of:

1. following the general precedent and allowing an attribute specifier sequence following the identifier (or `this` keyword) in a simple capture or init capture, or
2. following the precedent for function parameters and allowing an attribute specifier sequence both at the beginning of a capture declaration and following the identifier (or `this` keyword).

Option 1 is proposed based on the rationale that, since explicitly named capture declarations always include an identifier (or `this` keyword), the cognitive and implementation complexity of supporting multiple locations is not warranted.

If it is accepted that an attribute on a lambda capture declaration appertains to the notional non-static data member, then it follows that an attribute on a default lambda capture declaration appertains individually to each of the notional non-static data members of the implicit captures; this is consistent with attributes specified with function parameter pack declarations. This could be useful to, for example, apply the `[[no_unique_address]]` attribute to all implicit captures.

Default capture declarations consist solely of either the `&` or `=` token. The choices for an attribute specifier sequence are easy; the sequence either goes before the token, after the token, or at either location. For consistency with the proposed position for explicit captures, placing the attribute specifier sequence after the token is proposed.

Placing an attribute specifier sequence after `*this` could be a cause for confusion if programmers interpret it as applying to `this` as opposed to the intended meaning of it applying to the captured entity, the object denoted by `*this`. The remedy for such potential confusion would be to permit an attribute specifier sequence only at the beginning of each capture declaration.

Proposal

The following grammar changes are proposed:

- For namespace alias definitions:
 - Allow an attribute specifier sequence to appear immediately after the namespace keyword (consistent with namespace definitions).
- For type template parameters and template template parameters:
 - Allow an attribute specifier sequence to appear at the beginning of the parameter declaration (consistent with non-type template parameter and function parameter declarations).

- Allow an attribute specifier sequence to appear following an identifier in the parameter declaration (consistent with non-type template parameter and function parameter declarations).
- For lambda captures:
 - Allow an attribute specifier sequence to appear following an identifier in an explicit capture declaration, but not at the beginning of one (consistent with the general rule for attribute specifier sequences).
 - Allow an attribute specifier sequence to appear following `this` or `*this` in an explicit capture declaration, but not at the beginning of one.
 - Allow an attribute specifier sequence to appear following `&` or `=` in a capture default declaration, but not at the beginning of one.

The following attribute appertainment changes are proposed:

- Allow the `[[deprecated]]` attribute to appertain to a namespace alias.
- Allow the `[[maybe_unused]]` attribute to appertain to the non-static data member, if any, corresponding to a lambda capture.
- Allow the `[[maybe_unused]]` attribute to appertain to type template parameters and template template parameters (in addition to non-type template parameters).
- Allow the `[[no_unique_address]]` attribute to appertain to the non-static data member, if any, corresponding to a lambda capture.

Taken together, these changes allow for the following:

```
namespace ns {}
namespace [[deprecated]] nsa1 = ns; // Ok, previously ill-formed.
namespace nsa2 [[deprecated]] = ns; // Ill-formed; invalid attribute location.

template<[[maybe_unused]] int nttp> // Ok.
void ft1();
template<int nttp [[maybe_unused]]> // Ok.
void ft2();
template<[[maybe_unused]] typename ttp> // Ok, previously ill-formed.
void ft3();
template<typename ttp [[maybe_unused]]> // Ok, previously ill-formed.
void ft4();
template<[[maybe_unused]] template<typename> class ttp> // Ok, previously ill-formed.
void ft5();
template<template<typename> class ttp [[maybe_unused]]> // Ok, previously ill-formed.
void ft6();

struct S {
    void mf(int p) {
        [ [[maybe_unused]] = ]{}(); // Ill-formed; invalid attribute location.
        [ = [[maybe_unused]] ]{}(); // Ok but pointless [[maybe_unused]], previously ill-formed.
        [ [[maybe_unused]] & ]{}(); // Ill-formed; invalid attribute location.
        [ & [[maybe_unused]] ]{}(); // Ok but pointless [[maybe_unused]], previously ill-formed.
        [ [[maybe_unused]] p ]{}(); // Ill-formed; invalid attribute location.
        [ p [[maybe_unused]] ]{}(); // Ok, previously ill-formed.
        [ [[maybe_unused]] this ]{}(); // Ill-formed; invalid attribute location.
        [ this [[maybe_unused]] ]{}(); // Ok, previously ill-formed.
        [ [[maybe_unused]] *this ]{}(); // Ill-formed; invalid attribute location.
        [ *this [[maybe_unused]] ]{}(); // Ok, previously ill-formed.

        [ [[no_unique_address]] = ]{}(); // Ill-formed; invalid attribute location.
        [ = [[no_unique_address]] ]{}(); // Ok, previously ill-formed.
        [ [[no_unique_address]] & ]{}(); // Ill-formed; invalid attribute location.
        [ & [[no_unique_address]] ]{}(); // Ok, previously ill-formed.
        [ [[no_unique_address]] p ]{}(); // Ill-formed; invalid attribute location.
        [ p [[no_unique_address]] ]{}(); // Ok, previously ill-formed.
        [ [[no_unique_address]] this ]{}(); // Ill-formed; invalid attribute location.
        [ this [[no_unique_address]] ]{}(); // Ok, previously ill-formed.
        [ [[no_unique_address]] *this ]{}(); // Ill-formed; invalid attribute location.
        [ *this [[no_unique_address]] ]{}(); // Ok, previously ill-formed.
    }
};
```

Implementation experience

None.

Acknowledgements

Thanks to Aaron Ballman for vetting my expectation that attributes should be allowed on approximately everything and for providing feedback for multiple drafts of this proposal.

References

- [N3760] "[[deprecated]] attribute", N3760, 2013.
<https://wg21.link/n3760>
- [N4988] "Working Draft, Standard for Programming Language C++", N4988, 2024.
<https://wg21.link/n4988>
- [P0068R0] "Proposal of [[unused]], [[nodiscard]] and [[fallthrough]] attributes.", P0068R0, 2015.
<https://wg21.link/p0068r0>
- [P0212R1] "Wording for [[maybe_unused]] attribute.", P0212R1, 2016.
<https://wg21.link/p0212r1>
- [P0840R2] "Language support for empty objects", P0840R2, 2018.
<https://wg21.link/p0840r2>

Wording

These changes are relative to [N4988](#) [\[N4988\]](#).

- ☐ Hide inserted text
☐ Hide deleted text

Modify [\[expr.prim.lambda.capture\]](#):

```
lambda-capture:
  capture-default
  capture-list
  capture-default , capture-list

capture-default:
  & attribute-specifier-seqopt
  = attribute-specifier-seqopt

capture-list:
  capture
  capture-list , capture

capture:
  simple-capture
  init-capture

simple-capture:
  identifierattributed-identifier ...opt
  & identifierattributed-identifier ...opt
```



```
this attribute-specifier-seqopt  
* this attribute-specifier-seqopt
```

init-capture:

```
...opt identifierattributed-identifier initializer  
& ...opt identifierattributed-identifier initializer
```

Add an additional paragraph following [\[expr.prim.lambda.capture\]](#), [paragraph 1](#):

The optional attribute-specifier-seq in a capture-default or simple-capture or in an attributed-identifier of a simple-capture or init-capture appertains to the unnamed non-static data member, if any, declared in the closure type for the captured entity (see below).

Modify [\[namespace.alias\]](#), [paragraph 1](#):

A namespace-alias-definition declares an alternate name for a namespace according to the following grammar:

namespace-alias:
identifier

namespace-alias-definition:
namespace attribute-specifier-seq_{opt} identifier = qualified-namespace-specifier ;

qualified-namespace-specifier:
nested-name-specifier_{opt} namespace-name

Add an additional paragraph following [\[namespace.alias\]](#), [paragraph 2](#):

The optional attribute-specifier-seq_{opt} in a namespace-alias-definition appertains to the namespace alias being defined.

Modify [\[dcl.attr.deprecated\]](#), [paragraph 2](#):

The attribute may be applied to the declaration of a class, a typedef-name, a variable, a non-static data member, a function, a namespace, a namespace alias, an enumeration, an enumerator, a concept, or a template specialization.

Modify [\[dcl.attr.unused\]](#), [paragraph 2](#):

The attribute may be applied to the declaration of a class, typedef-name, variable (including a structured binding declaration), structured binding, non-static data member, lambda capture, function, function parameter, template parameter, enumeration, or enumerator, or to an identifier label ([\[stmt.label\]](#)).

Modify [\[dcl.attr.nuniqueaddr\]](#), [paragraph 1](#):

The attribute-token `no_unique_address` specifies that a non-static data member is a potentially-overlapping subobject ([\[intro.object\]](#)). No attribute-argument-clause shall be present. The attribute may appertain be applied to a lambda capture or a non-static data member other than a bit-field.

Modify [\[temp.param\]](#), [paragraph 1](#):

The syntax for template-parameters is:

template-parameter:
type-parameter
parameter-declaration

type-parameter:

*attribute-specifier-seq*_{opt} *type-parameter-key* ..._{opt} *identifier* *attributed-identifier*_{opt}
*attribute-specifier-seq*_{opt} *type-parameter-key* *identifier* *attributed-identifier*_{opt} = *type-id*
*attribute-specifier-seq*_{opt} *type-constraint* ..._{opt} *identifier* *attributed-identifier*_{opt}
*attribute-specifier-seq*_{opt} *type-constraint* *identifier* *attributed-identifier*_{opt} = *type-id*
*attribute-specifier-seq*_{opt} *template-head* *type-parameter-key* ..._{opt} *identifier* *attributed-identifier*_{opt}
*attribute-specifier-seq*_{opt} *template-head* *type-parameter-key* *identifier* *attributed-identifier*_{opt} = *id-expression*

type-parameter-key:

class
typename

type-constraint:

*nested-name-specifier*_{opt} *concept-name*
*nested-name-specifier*_{opt} *concept-name* < *template-argument-list*_{opt} >

The component names of a *type-constraint* are its *concept-name* and those of its *nested-name-specifier* (if any).

[Note 1: The > token following the *template-parameter-list* of a *type-parameter* can be the product of replacing a >> token by two consecutive > tokens ([temp.names]). — end note]

Add an additional paragraph following [temp.param], paragraph 2:

The optional *attribute-specifier-seq* in a *type-parameter* or in an *attributed-identifier* of a *type-parameter* appertains to the type parameter being declared.

Modify Annex A, "Grammar summary", [gram] to reflect the grammar changes to [expr.prim.lambda.capture], [namespace.alias], and [temp.param] shown above.